

FITBUDDY

AI FITNESS PLAN GENERATOR

PHASE 5 – PROJECT DEVELOPMENT

Implementation / Source Code Evidence Document

This updated document adds the supplied Visual Studio Code screenshots to the Phase 5 implementation evidence. The GitHub Repository Upload Format guide specifies that Phase 5 is the actual GitHub source-code submission; this PDF is supporting evidence.

Workflow: User Input → FitBuddy Web Interface → Gemini AI → Generated Fitness Plan

Backend: Python / FastAPI **AI:** Google Gemini **Database:** SQLite / SQLAlchemy

1. Project Development Overview

Phase 5 represents the implementation stage of FitBuddy. The project is a web-based AI fitness plan generator. The implementation uses a web interface to collect user fitness information, a Python/FastAPI backend to process the request, Google Gemini AI to generate the fitness plan, and SQLite with SQLAlchemy to store user and plan information.

The implemented workflow is: User Input → FitBuddy Web Interface → Gemini AI → Generated Fitness Plan. The application also supports nutrition/recovery guidance and feedback-based plan updates.

2. Technologies Used

Technology	Purpose
HTML	Web application structure and user interface
CSS	Interface styling and layout
JavaScript	Web interactions
Python	Backend application development
FastAPI	Backend framework and API endpoints
Google Gemini API	AI-based fitness plan generation
SQLite	Database storage
SQLAlchemy	Database interaction / ORM
Swagger UI	API testing and documentation

3. Implemented Application Modules

3.1 User Input Module

Collects user information including name, age, weight, fitness goal, intensity and preferences. The submitted information is passed to the backend for plan generation.

3.2 AI Fitness Plan Generation Module

The backend converts the user's information into an AI prompt and sends it to Google Gemini. Gemini generates a personalized workout plan.

3.3 Workout Plan Module

Displays the generated workout schedule, including the structured 7-day plan and exercise-related information.

3.4 Nutrition and Recovery Module

The implementation generates a nutrition/recovery tip along with the workout plan and displays it to the user.

3.5 Feedback Module

The user can submit feedback about the generated plan. The system retrieves the original plan and uses the feedback with Gemini to generate an updated 7-day plan.

3.6 Plan Display Module

Displays the generated or updated plan in a structured and readable format.

4. Backend Implementation

The main application route is **/generate-workout**. It accepts user form data such as name, age, weight, goal, intensity and preferences, and uses a database session while processing the request. The backend calls the workout-plan generation logic and separately generates a nutrition/recovery tip. The resulting information is stored and returned for display.

5. AI Integration

Google Gemini AI is integrated into FitBuddy for personalized fitness-plan generation. The user's input is supplied to the Gemini model to generate a structured workout plan and nutrition/recovery guidance. The feedback workflow also sends the original plan together with the user's feedback to Gemini, which produces an updated 7-day plan.

6. Database Implementation

FitBuddy uses SQLite with SQLAlchemy. The implementation stores user information, generated workout plans, nutrition/recovery tips, feedback and updated plans.

7. Development Evidence

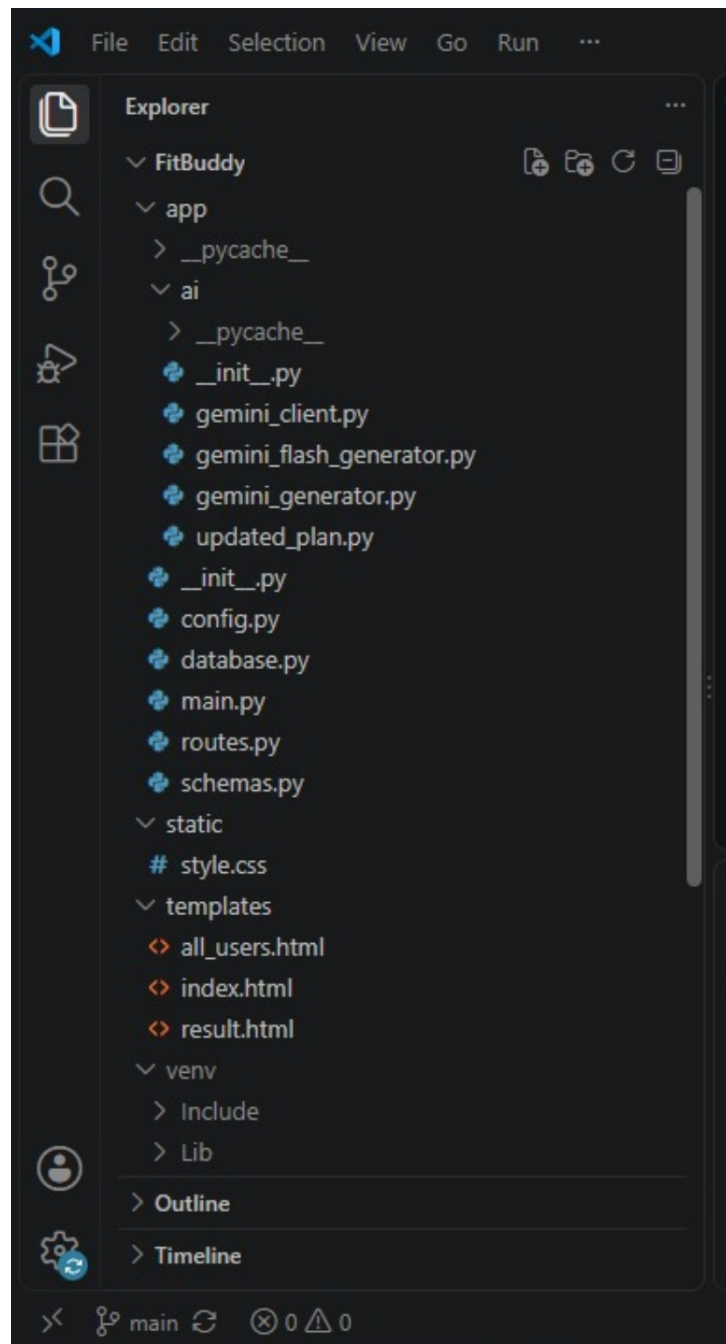
- Working backend/API implementation using Python and FastAPI.
- Gemini AI integration for personalized workout-plan generation.
- SQLite database integration using SQLAlchemy.
- 7-day workout-plan generation and display.
- Nutrition and recovery tip generation.
- Feedback submission and updated-plan generation.
- Swagger UI used for API testing and documentation.

8. Phase 5 GitHub Upload Method

The second uploaded guide states that Project Development (Phase 5) is where the actual GitHub source code belongs. The complete project/repository should be uploaded rather than only a descriptive document.

- Application/backend source – core application implementation
- API / route implementation – request handling and workout generation
- Database implementation – SQLite / SQLAlchemy data storage
- AI / recommendation implementation – Gemini-based plan generation
- Frontend files – user input and generated-plan display
- requirements.txt – project software dependencies
- Other project configuration files required for application setup

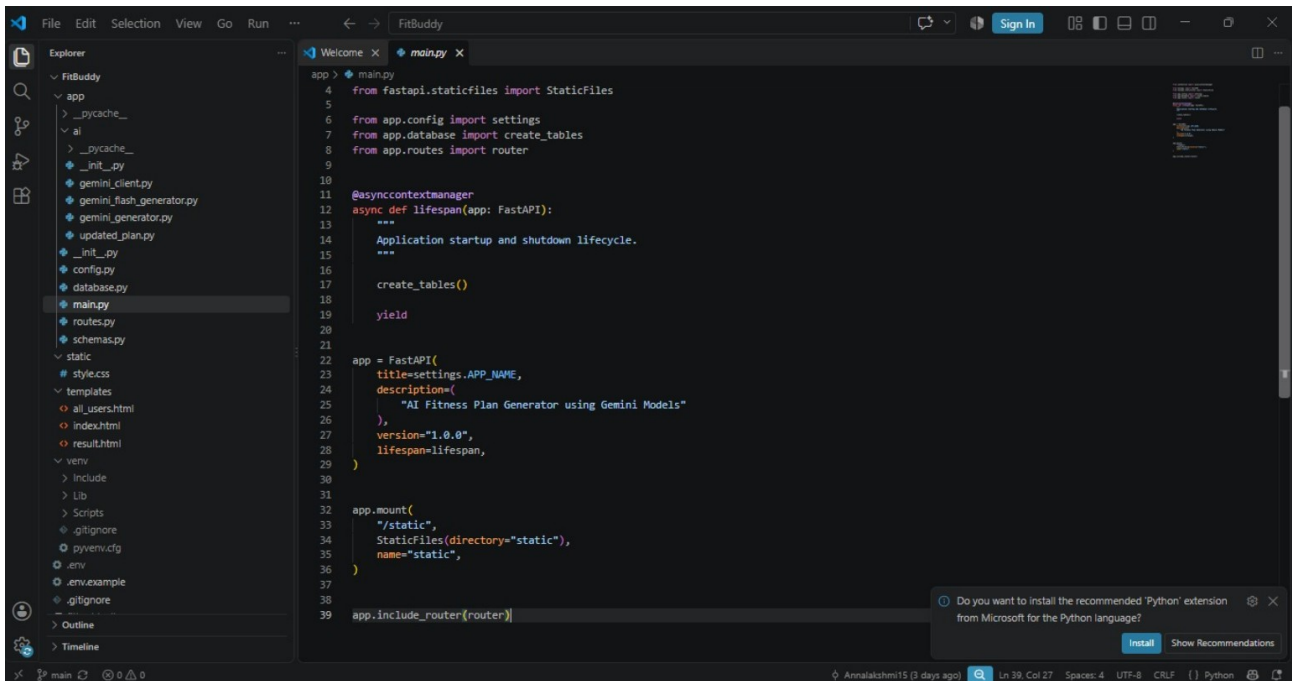
Security: Do not upload a real .env containing passwords, API keys or other secrets. Use .env.example for submission.



Screenshot 2 – Expanded app/ai module and project structure

10. FastAPI Source Code – main.py

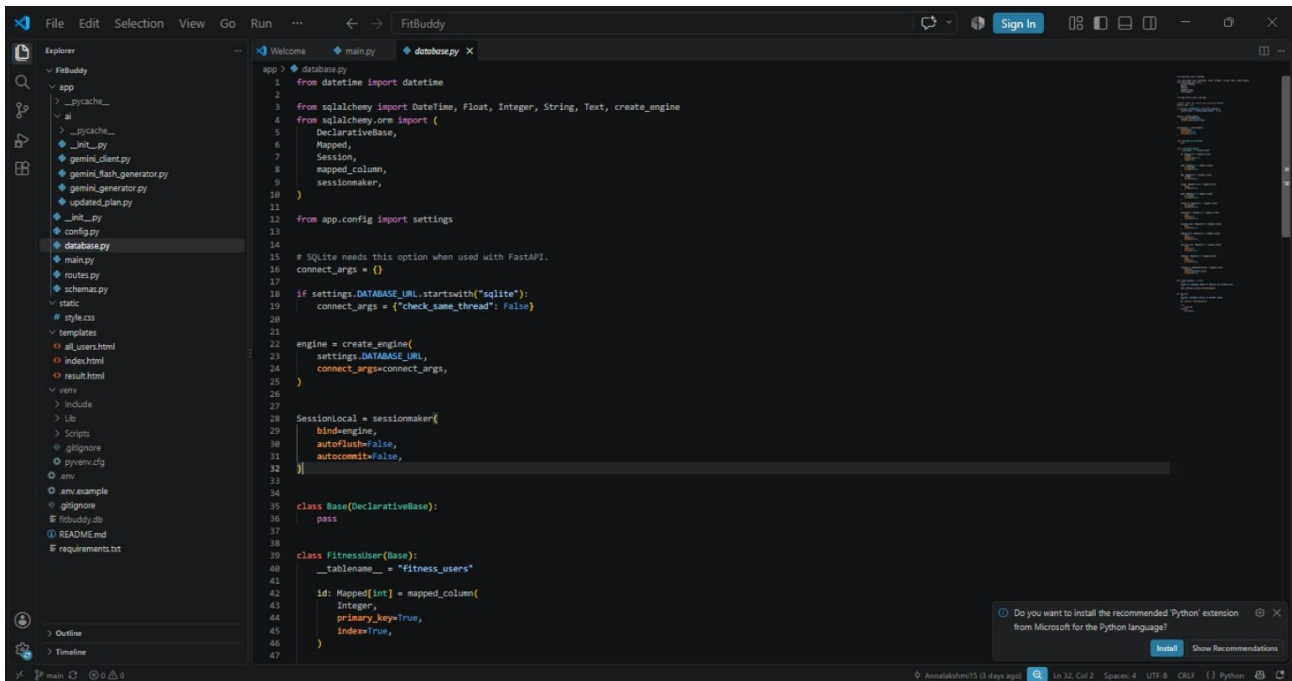
The supplied main.py screenshot shows FastAPI application setup, application lifecycle, database table creation, application metadata, static-file mounting and router inclusion.



Screenshot 3 – main.py FastAPI application implementation

11. AI Generator Source Code

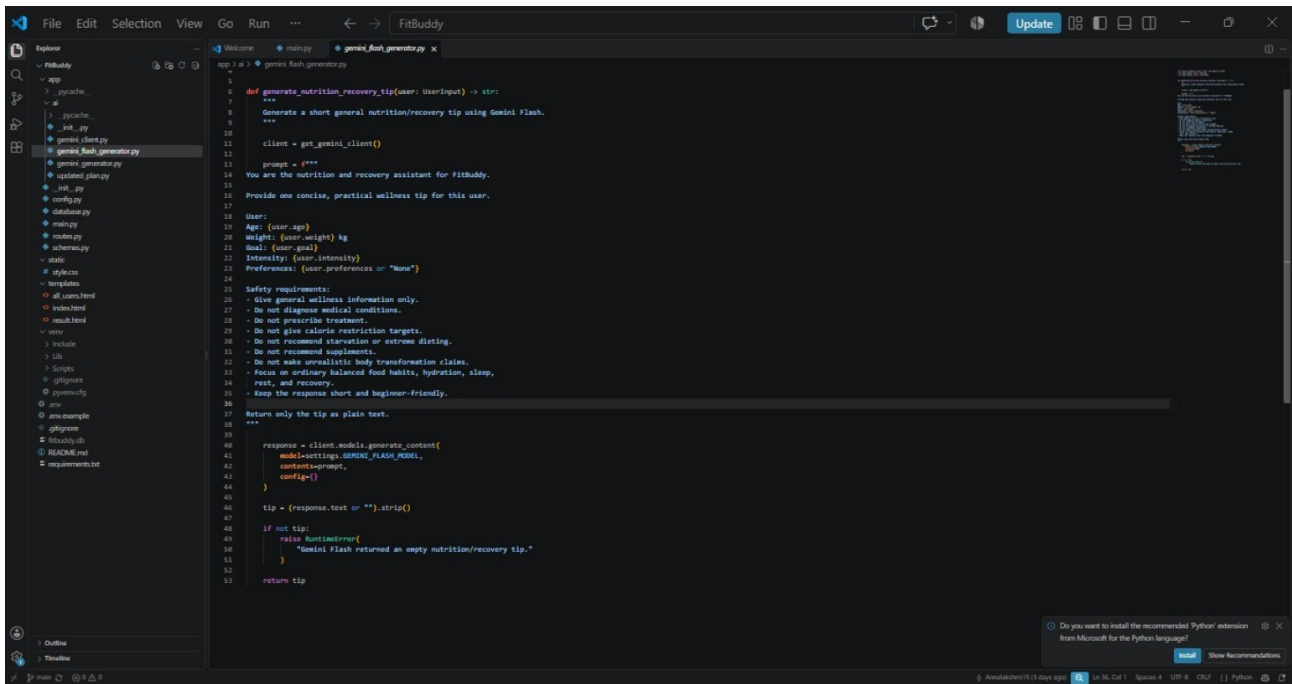
The supplied screenshot shows the Gemini Flash generator implementation, including the prompt, user information, safety requirements and model response handling.



Screenshot 6 – AI generator implementation in the app/ai module

12. Database Source Code – database.py

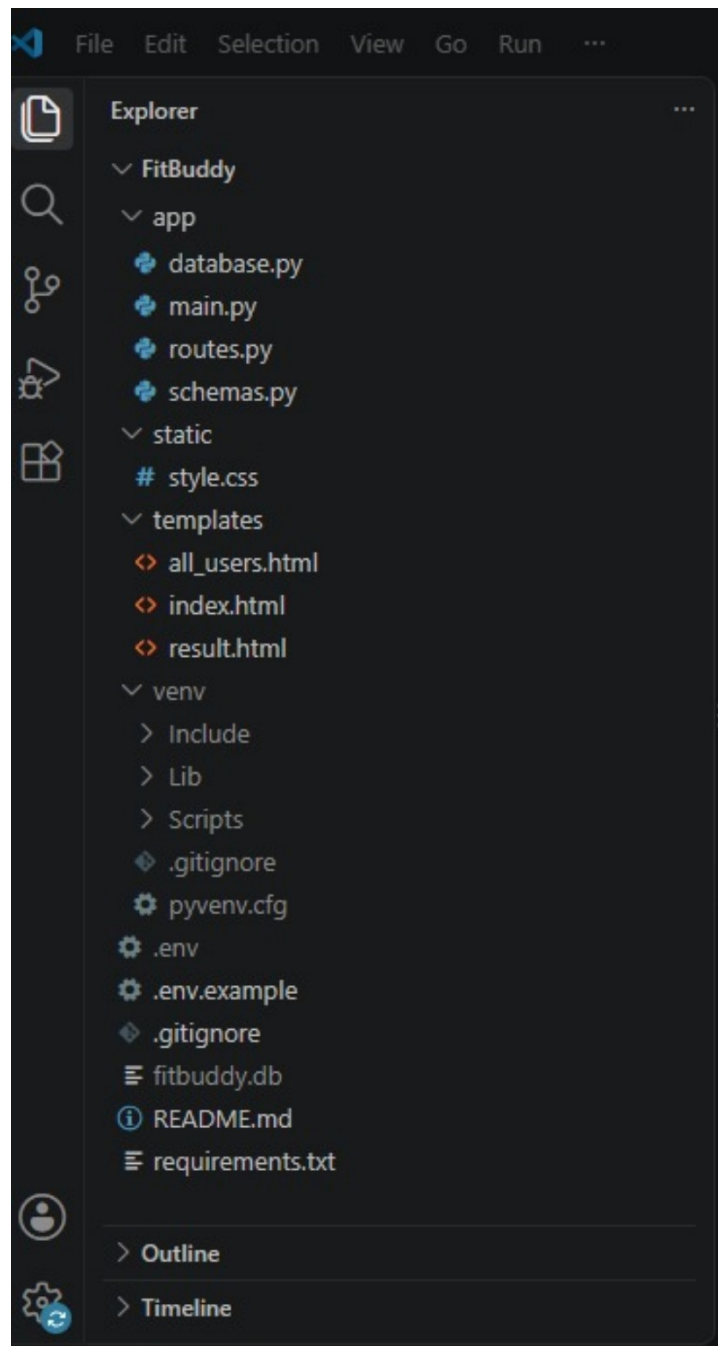
The supplied database.py screenshot shows SQLite/SQLAlchemy engine setup, session configuration and the database model implementation.



Screenshot 5 – database.py SQLite/SQLAlchemy implementation

13. Configuration and Repository Evidence

This screenshot shows the repository-level configuration and supporting files, including templates, static CSS, README.md, requirements.txt, environment example and the local database file.



Screenshot 4 – Project repository structure including configuration and dependency files

14. Phase 5 Submission Checklist

Item	Phase 5 use
Backend source	Core application implementation
API / routes	Request handling and workout generation
Database files	SQLite / SQLAlchemy storage
AI module	Gemini-based plan generation
Frontend files	User input and generated-plan display
requirements.txt	Software dependencies
.env.example	Safe configuration template; no real secrets
README.md	Project setup / usage documentation

15. Implementation Result

The supplied project documentation reports that FitBuddy successfully generates AI-based fitness plans from user inputs. The generated result contains a 7-day workout schedule, exercise duration, instructions, recovery suggestions and a nutrition/recovery tip. The application also supports feedback and generation of an updated workout plan.

16. Phase 5 Conclusion

The FitBuddy Project Development Phase demonstrates the implementation of the application's core workflow using Python/FastAPI, Google Gemini AI, SQLite/SQLAlchemy, HTML, CSS and JavaScript. The implementation connects user input, AI plan generation, database storage, nutrition/recovery guidance and feedback-based plan updating into the working application.

Submission note: Use the complete GitHub source-code repository as the actual Phase 5 upload. This PDF can be attached as supporting implementation evidence.